

"""

UMA Equalities – static fixed-point constraints on the projected field state.

These are the Constraint Alpha / Beta / Gamma operators from the Semantic Engine formulation: equality markers that keep the system anchored to its core architecture regardless of differential noise.

A constraint $c(z) = 0$ is enforced by a projection step after each filter update. Three types are supported:

LinearConstraint: $a^T z = b$ (hyperplane anchor)
BallConstraint: $\|P z\| \leq r$ (norm boundary)
QuadraticConstraint: $z^T Q z \leq \kappa$ (ellipsoidal density anchor)

Each implements:

.check(z) -> float residual $|c(z)|$; 0 = fully satisfied
.project(z) -> z_proj nearest z satisfying the constraint
.as_dict() -> metadata for logging

The ConstraintSet bundles multiple equalities and applies them in sequence (alternating projection, Dykstra's algorithm, or simple cascade).

Constraint semantics in the Semantic Engine:

Alpha – dIse: Defines the static boundary of the product's semantic space.
→ LinearConstraint on a user-chosen projection of z .
Beta – V:[H: Establishes structural compilation boundary.
→ BallConstraint on $\|z\|$: keeps state within the "compiled" region of the Fourier manifold.
Gamma – Jd\$m4: Fixes data density for metric re-articulation.
→ QuadraticConstraint: ellipsoidal metric anchor derived from the empirical Fisher information.

"""

```
from __future__ import annotations
import numpy as np
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import List, Optional, Tuple, Dict, Any
```

```
# -----
# Abstract base
# -----
```

```

class EqualityConstraint(ABC):
    """Abstract fixed-point constraint on  $z \in \mathbb{R}^d$ ."""

    def __init__(self, name: str):
        self.name = name
        self._violation_history: List[float] = []

    @abstractmethod
    def check(self, z: np.ndarray) -> float:
        """Return residual  $\geq 0$ ; 0 means  $z$  exactly satisfies the constraint."""

    @abstractmethod
    def project(self, z: np.ndarray) -> np.ndarray:
        """Return nearest  $z'$  satisfying the constraint (projection onto feasible

    def apply(self, z: np.ndarray) -> Tuple[np.ndarray, float]:
        """Project and record violation before projection."""
        viol = self.check(z)
        self._violation_history.append(viol)
        z_proj = self.project(z)
        return z_proj, viol

    @property
    def violation_history(self) -> np.ndarray:
        return np.array(self._violation_history)

    @property
    def is_satisfied(self) -> bool:
        """True if last recorded violation is below  $1e-6$ ."""
        if not self._violation_history:
            return False
        return self._violation_history[-1] < 1e-6

    def as_dict(self) -> Dict[str, Any]:
        return {
            "name": self.name,
            "type": type(self).__name__,
            "n_applications": len(self._violation_history),
            "last_violation": self._violation_history[-1] if self._violation_hist
            "satisfied": self.is_satisfied,
        }

    def reset(self) -> None:
        self._violation_history.clear()

```

```

# Constraint Alpha - LinearConstraint (Constraint dIse)
# -----

class LinearConstraint(EqualityConstraint):
    """
    Hyperplane equality:  $a^T z = b$ .

    Semantic role (Constraint Alpha / dIse):
        Defines the static boundary of the product's semantic space.
         $z$  must lie on the hyperplane normal to  $a \in \mathbb{R}^d$  at signed distance  $b$ .

    Projection:
         $z_{\text{proj}} = z - ((a^T z - b) / \|a\|^2) a$  (orthogonal projection)
    """

    def __init__(self, a: np.ndarray, b: float, name: str = "alpha_dIse"):
        super().__init__(name)
        a = np.asarray(a, dtype=float)
        norm_sq = float(a @ a)
        if norm_sq < 1e-12:
            raise ValueError("Constraint normal vector `a` must be nonzero.")
        self._a = a
        self._b = b
        self._norm_sq = norm_sq

    @classmethod
    def from_direction(cls, d: int, direction: int = 0, b: float = 0.0,
                      name: str = "alpha_dIse") -> "LinearConstraint":
        """Convenience: constrain  $z[\text{direction}] = b$ ."""
        a = np.zeros(d)
        a[direction] = 1.0
        return cls(a, b, name)

    def check(self, z: np.ndarray) -> float:
        return abs(float(self._a @ z) - self._b)

    def project(self, z: np.ndarray) -> np.ndarray:
        residual = float(self._a @ z) - self._b
        return z - (residual / self._norm_sq) * self._a

# -----
# Constraint Beta - BallConstraint (Constraint V:[H])
# -----

class BallConstraint(EqualityConstraint):
    """

```

ℓ_2 -ball inequality (converted to equality at boundary): $\|P z\| \leq r$.

Semantic role (Constraint Beta / V:[H]):

Establishes the structural compilation boundary – keeps the projected state within the "compiled" region of the Fourier manifold.

Projection:

If $\|P z\| \leq r$: z unchanged (feasible).

If $\|P z\| > r$: $z_{\text{proj}} = z_{\text{fixed}} + r * (P z) / \|P z\|$
where z_{fixed} is the component of z outside range(P).

P may be None (= identity), in which case the constraint is on the full $\|z\|$. P may also be a projection matrix ($P^2 = P$, $P^T = P$) for constrained subspaces.

"""

```
def __init__(self, r: float, P: Optional[np.ndarray] = None,
              center: Optional[np.ndarray] = None,
              name: str = "beta_VH"):
    super().__init__(name)
    if r <= 0:
        raise ValueError("Ball radius r must be positive.")
    self._r = r
    self._P = P # None → identity
    self._center = center # None → 0

def _pz(self, z: np.ndarray) -> np.ndarray:
    v = z if self._center is None else z - self._center
    return v if self._P is None else self._P @ v

def check(self, z: np.ndarray) -> float:
    norm = float(np.linalg.norm(self._pz(z)))
    return max(0.0, norm - self._r)

def project(self, z: np.ndarray) -> np.ndarray:
    center = np.zeros_like(z) if self._center is None else self._center
    v = z - center
    if self._P is None:
        norm = float(np.linalg.norm(v))
        if norm <= self._r:
            return z
        return center + self._r * v / norm
    # P is a projector: v = P v + (I-P) v
    Pv = self._P @ v
    norm_Pv = float(np.linalg.norm(Pv))
    if norm_Pv <= self._r:
        return z
```

```

    Pv_proj = self._r * Pv / norm_Pv
    complement = v - Pv # (I-P) v, unchanged by constraint
    return center + Pv_proj + complement

# -----
# Constraint Gamma - QuadraticConstraint (Constraint Jd$m4)
# -----

class QuadraticConstraint(EqualityConstraint):
    """
    Ellipsoidal constraint:  $z^T Q z \leq \kappa$ .

    Semantic role (Constraint Gamma / Jd$m4):
        Fixes data density for metric re-articulation. Q encodes the
        Fisher information geometry (the "metric re-articulation" operator).
        The ellipsoid  $\{z : z^T Q z \leq \kappa\}$  is the admissible region at
        confidence level  $\kappa$ .

    Projection:
        Via eigendecomposition of Q:  $Q = V \Lambda V^T$ .
        In the eigenbasis  $w = V^T z$ , the ellipsoid becomes  $\|\Lambda^{\{1/2\}} w\| \leq \sqrt{\kappa}$ .
        Binary search on the Lagrange multiplier  $\lambda$ : project
             $w \rightarrow w / (1 + \lambda \Lambda_i^{-1})$ 
        until the ellipsoidal constraint is met.
        (This is the standard quadratic projection via KKT conditions.)

    Q must be SPD or at least PSD. If Q is None, defaults to identity
    (ball of radius  $\sqrt{\kappa}$  centered at origin).
    """

    def __init__(self, kappa: float, Q: Optional[np.ndarray] = None,
                  center: Optional[np.ndarray] = None,
                  name: str = "gamma_Jd4m"):
        super().__init__(name)
        if kappa <= 0:
            raise ValueError("kappa must be positive.")
        self._kappa = kappa
        self._center = center
        self._Q = Q
        # Cache eigendecomposition
        if Q is not None:
            eigs, vecs = np.linalg.eigh(0.5 * (Q + Q.T))
            self._eigs = np.clip(eigs, 1e-12, None)
            self._vecs = vecs # (d, d), columns are eigenvectors
        else:
            self._eigs = None

```

```

        self._vecs = None

def _mahal(self, z: np.ndarray) -> float:
    v = z if self._center is None else z - self._center
    if self._Q is None:
        return float(v @ v)
    return float(v @ self._Q @ v)

def check(self, z: np.ndarray) -> float:
    return max(0.0, self._mahal(z) - self._kappa)

def project(self, z: np.ndarray) -> np.ndarray:
    center = np.zeros_like(z) if self._center is None else self._center
    v = z - center
    if self._mahal(z) <= self._kappa:
        return z # already feasible

    if self._Q is None:
        # Identity case: ball projection
        norm = float(np.linalg.norm(v))
        return center + np.sqrt(self._kappa) * v / (norm + 1e-15)

    # Ellipsoidal projection via KKT
    #  $w = V^T v$  in eigenbasis
    w = self._vecs.T @ v # (d,)
    lam_vals = self._eigs # eigenvalues of  $Q \geq 0$ 
    kappa = self._kappa

    # Binary search for Lagrange multiplier  $\mu \geq 0$ :
    #  $w_{\text{proj}_i} = w_i / (1 + \mu * \text{lam\_vals}_i)$ 
    # constraint:  $\sum (\text{lam\_vals}_i * w_{\text{proj}_i}^2) = \kappa$ 

    def constraint_val(mu):
        w_proj = w / (1 + mu * lam_vals)
        return float(np.sum(lam_vals * w_proj ** 2)) - kappa

    # Bracket: mu=0 gives  $\sum \text{lam } w^2 > \kappa$  (infeasible at mu=0 by assumption)
    mu_lo, mu_hi = 0.0, 1e8
    for _ in range(60):
        mu_mid = 0.5 * (mu_lo + mu_hi)
        val = constraint_val(mu_mid)
        if val > 0:
            mu_lo = mu_mid
        else:
            mu_hi = mu_mid
    if mu_hi - mu_lo < 1e-12:
        break

```

```

        mu_opt = 0.5 * (mu_lo + mu_hi)
        w_proj = w / (1 + mu_opt * lam_vals)
        v_proj = self._vecs @ w_proj
        return center + v_proj

# -----
# ConstraintSet - bundles Alpha, Beta, Gamma and applies them in sequence
# -----

class ConstraintSet:
    """
    Bundle of UMA Equalities applied in cascade (alternating projection).

    For k constraints C1, ..., Ck, applies them in round-robin until
    the total violation drops below `tol` or `max_iters` is reached.

    Usage::

        cs = ConstraintSet([alpha, beta, gamma], max_iters=20, tol=1e-6)
        z_anchored, report = cs.apply(z_raw)
    """

    def __init__(
        self,
        constraints: List[EqualityConstraint],
        max_iters: int = 20,
        tol: float = 1e-6,
    ):
        self.constraints = constraints
        self.max_iters = max_iters
        self.tol = tol

    def apply(self, z: np.ndarray) -> Tuple[np.ndarray, Dict[str, Any]]:
        """
        Project z onto the intersection of all constraints.

        Returns (z_anchored, report_dict).
        """
        z_curr = z.copy()
        violations_per_iter = []
        for it in range(self.max_iters):
            total_viol = 0.0
            for c in self.constraints:
                z_curr, viol = c.apply(z_curr)
                total_viol += viol

```

```

        violations_per_iter.append(total_viol)
        if total_viol < self.tol:
            break

    report = {
        "n_iters": len(violations_per_iter),
        "final_total_violation": violations_per_iter[-1] if violations_per_iter else 0,
        "converged": violations_per_iter[-1] < self.tol if violations_per_iter else False,
        "constraint_reports": [c.as_dict() for c in self.constraints],
    }
    return z_curr, report

def reset(self) -> None:
    for c in self.constraints:
        c.reset()

@classmethod
def default_semantic_constraints(
    cls,
    dim: int,
    z_target: np.ndarray,
    ball_radius_factor: float = 3.0,
    kappa_factor: float = 9.0,
    alpha_direction: int = 0,
) -> "ConstraintSet":
    """
    Build the canonical three-constraint set (Alpha, Beta, Gamma)
    anchored to a given z_target.

    Alpha (dIse):  z[alpha_direction] = z_target[alpha_direction]
    Beta  (V:[H]):  ||z - z_target|| ≤ r  where r = ball_radius_factor * ||z_target||
    Gamma (Jd$m4):  (z - z_target)T I (z - z_target) ≤ κ
                    (unit-metric ellipsoid, radius √κ around target)
    """
    # Alpha
    a = np.zeros(dim)
    a[alpha_direction] = 1.0
    b = float(z_target[alpha_direction])
    alpha = LinearConstraint(a, b, name="alpha_dIse")

    # Beta
    r = ball_radius_factor * (float(np.linalg.norm(z_target)) + 1.0)
    beta = BallConstraint(r=r, center=z_target, name="beta_VH")

    # Gamma: ellipsoidal anchor, κ = kappa_factor * dim
    kappa = kappa_factor * dim
    gamma = QuadraticConstraint(kappa=kappa, Q=None, center=z_target,

```



```
name="gamma_Jd4m")
```

```
return cls([alpha, beta, gamma])
```